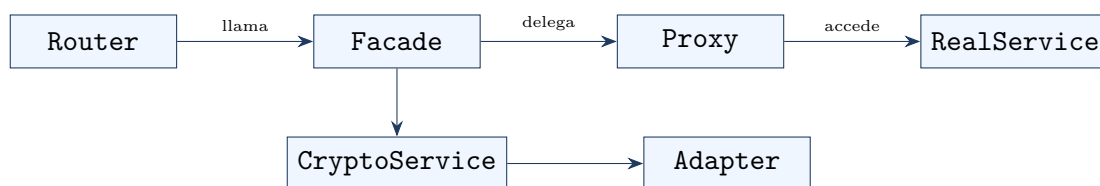


Patrones Estructurales Implementados

Plataforma Voz del Ciudadano

Los patrones estructurales describen cómo componer clases y objetos para formar estructuras más grandes, manteniendo la flexibilidad del sistema. Se enfocan en las relaciones entre entidades y cómo estas se conectan. En este proyecto se implementaron cinco patrones estructurales, cada uno resolviendo un problema concreto del dominio.

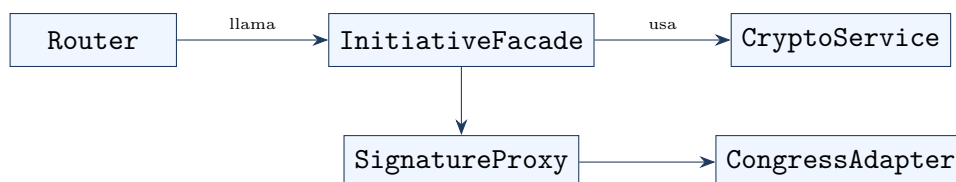
El siguiente diagrama muestra el flujo completo cuando un ciudadano firma una propuesta, pasando por todos los patrones involucrados:



1. Facade `patterns/facade.py`

El ciclo de vida de una propuesta involucra múltiples subsistemas: persistencia en base de datos, comprobaciones criptográficas, notificación al Congreso y registro de auditoría. Coordinarlos directamente desde el router generaría alto acoplamiento y código difícil de mantener.

`InitiativeFacade` actúa como punto de entrada único. El router solo llama a `create_initiative`, `publish_initiative` o `sign_initiative`; la Facade orquesta internamente el Proxy, el Decorator y el Adapter sin exponerlos.



```

1  # El router solo llama a la Facade
2  facade = InitiativeFacade()
3
4  @router.post("/initiatives/{initiative_id}/sign")
5  def sign(initiative_id: str, req: SignRequest):
6      result = facade.sign_initiative(initiative_id, req.user_id)
7      return result
8
9  # Dentro de la Facade (orquestración de subsistemas)
10 def sign_initiative(self, initiative_id: str, user_id: str) -> dict:
11     initiative = store["initiatives"].get(initiative_id)
12     if initiative["status"] != "ACTIVA":
13         raise ValueError("La iniciativa no está activa")
14     user = store["users"].get(user_id)
15     if not user or not user.get("verified"):
16         raise PermissionError("El ciudadano debe estar verificado")
  
```

```

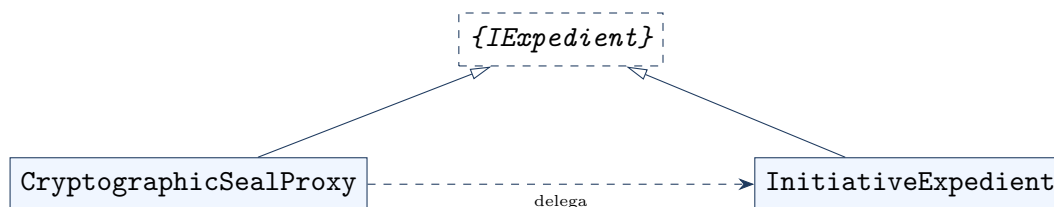
17     signature = self._signature_service.sign(user_id, initiative_id,
18         store)
19     initiative["signature_count"] += 1
20     if initiative["signature_count"] >= SIGNATURE_THRESHOLD:
21         self._seal_and_dispatch(initiative_id)
22     return signature
    
```

Listing 1: Facade: único punto de entrada desde el router

2. Proxy `patterns/proxy.py`

Antes de registrar una firma se deben cumplir tres reglas: la propuesta debe estar activa, el plazo de 90 días debe estar vigente y no debe haber sido firmada antes. Poner esta lógica dentro del servicio real lo contaminaría con responsabilidades de validación.

CryptographicSealProxy implementa la misma interfaz que InitiativeExpedient. Intercepta cada operación de escritura y verifica si el expediente ya fue sellado. Si alguna regla falla, lanza una excepción y el servicio real nunca se invoca.



```

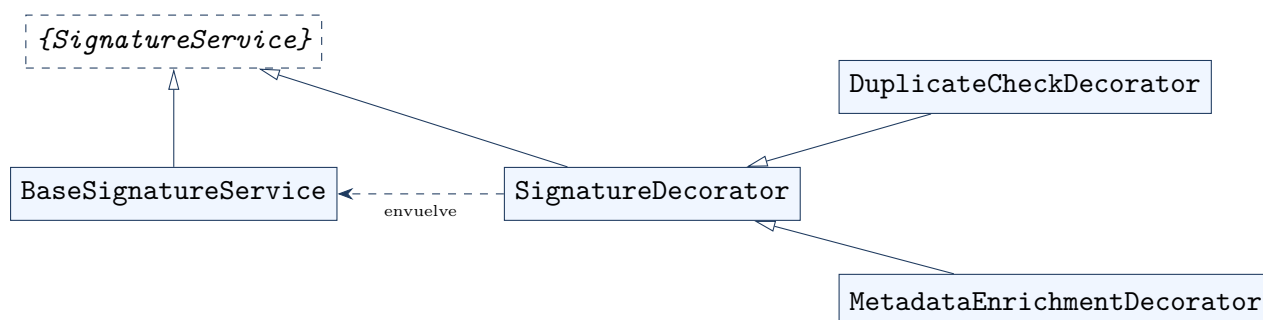
1 class CryptographicSealProxy:
2     def __init__(self, expedient: InitiativeExpedient):
3         self._expedient = expedient
4
5     def _is_sealed(self) -> bool:
6         return self._expedient.get().get("sealed", False)
7
8     def update(self, data: dict):
9         if self._is_sealed():
10             raise PermissionError(
11                 "El expediente está sellado. No se permiten
12                 modificaciones."
13             )
14         self._expedient.update(data)
15
16     def seal(self) -> str:
17         if self._is_sealed():
18             raise PermissionError("El expediente ya fue sellado
19             anteriormente")
20         content_str = json.dumps(self._expedient.get(), sort_keys=True)
21         seal_hash = hashlib.sha512(content_str.encode()).hexdigest()
22         self._expedient.update({
23             "sealed": True,
24             "seal_hash": seal_hash,
25             "sealed_at": datetime.now(timezone.utc).isoformat(),
26         })
27         return seal_hash
    
```

Listing 2: Proxy: intercepta escrituras sobre el expediente sellado

3. Decorator patterns/decorator.py

El servicio base de firma únicamente persiste el registro. Sin embargo, el sistema necesita además verificar que el ciudadano no haya firmado antes y enriquecer la firma con un hash SHA-256 del documento y un timestamp UTC. Estas son responsabilidades separadas que no deben mezclarse en una sola clase.

Dos decoradores se apilan sobre `BaseSignatureService`: primero `DuplicateCheckDecorator` verifica el duplicado, luego `MetadataEnrichmentDecorator` añade los campos de auditoría. Cada decorador puede añadirse o retirarse sin afectar al resto.



```

1 class DuplicateCheckDecorator(SignatureDecorator):
2     def sign(self, user_id: str, initiative_id: str, db: dict) -> dict:
3         already_signed = any(
4             s["user_id"] == user_id and s["initiative_id"] ==
5                 initiative_id
6             for s in db.get("signatures", {}).values()
7         )
8         if already_signed:
9             raise ValueError("El ciudadano ya firmó esta iniciativa")
10        return super().sign(user_id, initiative_id, db)
11
12 class MetadataEnrichmentDecorator(SignatureDecorator):
13     def sign(self, user_id: str, initiative_id: str, db: dict) -> dict:
14         sig = super().sign(user_id, initiative_id, db)
15         content = db["initiatives"].get(initiative_id, {}).get("content", "")
16         sig["doc_hash"] = hashlib.sha256(content.encode()).hexdigest()
17         sig["timestamp"] = datetime.now(timezone.utc).isoformat()
18         db["signatures"][sig["id"]] = sig
19         return sig
20
21 # Composición en la Facade (orden importa)
22 base = BaseSignatureService()
23 service = MetadataEnrichmentDecorator(DuplicateCheckDecorator(base))
    
```

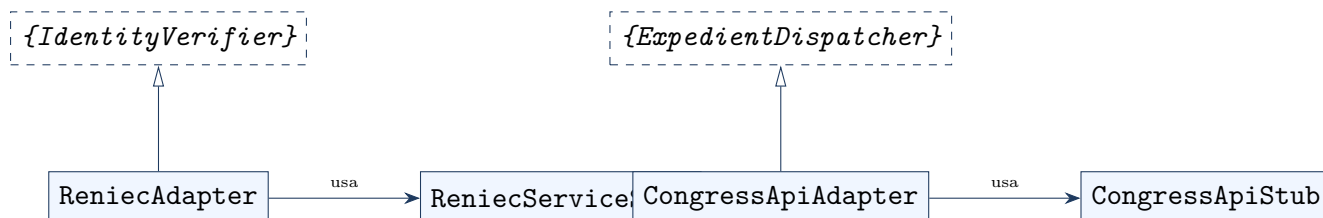
Listing 3: Decorator: cadena de validación y enriquecimiento

4. Adapter patterns/adapter.py

El sistema necesita consultar RENIEC para verificar la identidad de un ciudadano y enviar expedientes a la API del Congreso. Ambos servicios externos exponen interfaces propias

que no coinciden con lo que el dominio espera. Acoplar el dominio a esos formatos lo volvería frágil ante cambios externos.

`ReniecAdapter` traduce la llamada `verify(identifier)` al método `consultar_identidad(numero_documento)` del stub. `CongressApiAdapter` hace lo mismo para el envío del expediente. Ambos stub simulan los servicios reales y pueden reemplazarse por implementaciones reales sin cambiar el dominio.



```

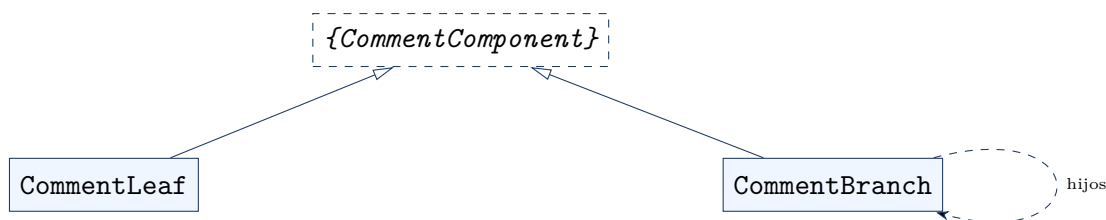
1 class ReniecServiceStub:
2     def consultar_identidad(self, numero_documento: str) -> dict:
3         es_valido = (
4             len(numero_documento) == 8
5             and numero_documento.isdigit()
6             and numero_documento != "00000000"
7         )
8         return {"valido": es_valido, "estado": "activo" if es_valido
9             else "inactivo"}
10
11 class ReniecAdapter(IdentityVerifier):
12     def __init__(self):
13         self._reniec = ReniecServiceStub()
14
15     def verify(self, identifier: str) -> bool:
16         result = self._reniec.consultar_identidad(identifier)
17         return result.get("valido", False) and result.get("estado") == "
18             activo"
19
20 class CongressApiAdapter(ExpedientDispatcher):
21     def send(self, initiative_id: str, content: dict) -> dict:
22         payload = {"id": initiative_id, "contenido": content, "formato":
23             "JSON-LD"}
24         return self._api.recibir_expediente(payload)
    
```

Listing 4: Adapter: traduce la interfaz del servicio externo

5. Composite [patterns/composite.py](#)

Los comentarios de una propuesta pueden recibir respuestas, y esas respuestas pueden recibir mas respuestas, formando una jerarquía de hasta 3 niveles. Tratar los comentarios hoja (sin hijos) distinto a los rama (con hijos) obligaría a condiciones en cada operación.

`CommentBranch` y `CommentLeaf` implementan la misma interfaz `CommentComponent`. El sistema serializa el árbol completo llamando `get_data()` en el nodo raíz, que se propaga recursivamente. La función `build_comment` decide que tipo de nodo crear según la profundidad y rechaza el nivel 4.



```

1 class CommentLeaf(CommentComponent):
2     def get_data(self) -> dict:
3         return {
4             "id": self.id, "text": self.text,
5             "level": self._depth, "children": [], "can_reply": False,
6         }
7
8 class CommentBranch(CommentComponent):
9     def add_child(self, comment: CommentComponent):
10         if comment.level() > MAX_DEPTH:
11             raise ValueError(f"Profundidad maxima ({MAX_DEPTH})
12                 alcanzada")
13         self._children.append(comment)
14
15     def get_data(self) -> dict:
16         return {
17             "id": self.id, "text": self.text,
18             "level": self._depth,
19             "children": [c.get_data() for c in self._children],
20             "can_reply": True,
21         }
22
23 def build_comment(text, author_id, parent_id=None, depth=1):
24     if depth > MAX_DEPTH:
25         raise ValueError(f"Profundidad maxima ({MAX_DEPTH}) alcanzada")
26     if depth == MAX_DEPTH:
27         return CommentLeaf(text, author_id, parent_id, depth)
28     return CommentBranch(text, author_id, parent_id, depth)
    
```

Listing 5: Composite: árbol de comentarios con profundidad maxima 3

Resumen

Patrón	Clase principal	Propósito
Facade	InitiativeFacade	Simplifica la orquestación de subsistemas
Proxy	CryptographicSealProxy	Controla el acceso al expediente sellado
Decorator	DuplicateCheckDecorator	Añade validación y metadata a la firma
Adapter	ReniecAdapter	Traduce interfaces externas incompatibles
Composite	CommentBranch/Leaf	Modela jerarquía de comentarios como árbol